

AI Capstone HW1

112550169 潘仰祐

1. Implementation

I modified the template instead of starting from zero.

Part 1. 2D semantic map construction

In `main.py`'s `main()` function, we can see TODO 1-2. Where I used color01.npy since the spec implies this is expected if no floating-point error occurs.

```
def main():
    """Entry point."""

    print("=== Step 1: Processing the 3D Map ===")
    # ===== TODO 1-2 =====
    map_img, occupancy_map, map_img_for_display, trans_info = load_and_filter_map(POINT_CLOUD_DATA, COLOR_DATA)
    POINT_CLOUD_DATA = "semantic_3d_pointcloud/point.npy"
    COLOR_DATA = "semantic_3d_pointcloud/color01.npy"
```

In `map\_processor.py`, I didn't use the colours to remove ceiling and floor but cut the height instead. Also, I set the map resolution to have 20 px per meter.

```
SCALE_FACTOR = 10000.0 / 255.0
# CEILING_COLOR = np.array([8, 255, 214])
# FLOOR_COLOR = np.array([255, 194, 7])
MAP_RESOLUTION = 20 # resolution: 5cm per pixel
```

Then, in TODO 1-1, I keep the space between $y = -1.3 \sim 0.3$. This can effectively cut the ceiling and floor (where $y = 0$ should be the sensor at 1.5m-height in the 2.5m-height floor). Also, this sorting is to reduce the noise of cushion points because I don't know how to remove them more better.

```
# 1. remove ceiling and floor by height
{# - real world is ~2.5m height per floor, ...
ceiling_threshold = 0.3
floor_threshold = -1.3

mask_ceiling = coords[:, 1] > ceiling_threshold
mask_floor = coords[:, 1] < floor_threshold
mask_remove = mask_ceiling | mask_floor

# 1.1. cut and sort
coords_cut = coords[~mask_remove]
colors_cut = colors[~mask_remove]
sort_idx = np.lexsort((1-coords_cut[:, 1],)) # sort by y
coords_cut = coords_cut[sort_idx]
colors_cut = colors_cut[sort_idx]
```

Then, a projection to the 2D space is done to form a map with semantic colours. The map is initially black-backgrounded.

```
# 2. project to 2D
# - map (x, z) to (u, v)
coords_cut_2d = coords_cut[:, [0, 2]]

# - The data itself:
#   - min_x = -0.77883, max_x = 0.15992
#   - min_z = -0.12620, max_z = 0.25277
min_u = np.min(coords_cut_2d[:, 0])
max_u = np.max(coords_cut_2d[:, 0])
min_v = np.min(coords_cut_2d[:, 1])
max_v = np.max(coords_cut_2d[:, 1])

# align top-left to origin
coords_cut_2d[:, 0] -= min_u
coords_cut_2d[:, 1] -= min_v

# 3. create map image
range_u = max_u - min_u
range_v = max_v - min_v
map_width = int(range_u * MAP_RESOLUTION) + 1
map_height = int(range_v * MAP_RESOLUTION) + 1

map_img = np.zeros((map_height, map_width, 3), dtype=np.float32) # pure black background
pixel_u = (coords_cut_2d[:, 0] * MAP_RESOLUTION).astype(int)
pixel_v = (coords_cut_2d[:, 1] * MAP_RESOLUTION).astype(int)
map_img[pixel_v, pixel_u] = colors_cut
```

Then, isolated points removal is done for areas < 20 since they are likely to be noises. Then, the background is made white.

```
# 4. remove isolated points
occupancy_mask = np.any(map_img > 0, axis=-1).astype(np.uint8)

# find connected color blocks and filter out small noise
num_labels, labels, stats, centroids = \
    cv2.connectedComponentsWithStats(
        occupancy_mask,
        connectivity=8 # 8: a 3x3 grid centered at the pixel
                       # vs. 4: up/down/left/right neighbours
    )

clean_mask = np.zeros_like(occupancy_mask)
for i in range(1, num_labels): # start from 1, skip 0 (background)
    if stats[i, cv2.CC_STAT_AREA] >= 20: # remove area < 20
        clean_mask[labels == i] = 1

# make the background + removed noises white
map_img[clean_mask == 0] = 1.0
```

But the isolated point removal accidentally remove some walls. So, we add manual patches (black lines) to prevent RRT explore outside later.

```
# 5. add patches
patches = [(159, 87, 160, 100), (91, 36, 93, 47), (137, 40, 153, 38)]
for u1, v1, u2, v2 in patches:
    # draw lines
    cv2.line(map_img, (u1, v1), (u2, v2), (0, 0, 0), 1)
    cv2.line(clean_mask, (u1, v1), (u2, v2), 1, 1)

# 6. inflate obstacles to get occupancy map
robot_radius_m = 0.15 # the agent is a 0.1m-radius pillar by default, bigger for safety
inflate_pixels = int(robot_radius_m * MAP_RESOLUTION)
kernel_size = inflate_pixels * 2 + 1
kernel_inflate = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (kernel_size, kernel_size))
occupancy_map = cv2.dilate(clean_mask, kernel_inflate, iterations=1)
```

After that, a occupancy map is drawn by obstacle inflation. Also, to visualize it, a map_img_for_selection is generated. (Despite this naming, I also use this image for path visualization later.) The projectional constant are also returned for unprojection from the pixel coordinate to the world later.

```
# 6. inflate obstacles to get occupancy map
robot_radius_m = 0.15 # the agent is a 0.1m-radius pillar by default, bigger for safety
inflate_pixels = int(robot_radius_m * MAP_RESOLUTION)
kernel_size = inflate_pixels * 2 + 1
kernel_inflate = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (kernel_size, kernel_size))
occupancy_map = cv2.dilate(clean_mask, kernel_inflate, iterations=1)

# extra edition:
# + add a visible inflation
# + convert RGB to BGR for cv2
map_img_for_selection = map_img.copy()
map_img_for_selection[clean_mask == 0] = np.stack([1 - 0.33 * occupancy_map] * 3, axis=-1)[clean_mask == 0]
map_img_for_selection = map_img_for_selection[:, :, [2, 1, 0]]

# return translation constants for TOD04 (pixel to world coordinate)
return map_img, occupancy_map.astype(np.float32), map_img_for_selection, (min_u, min_v, MAP_RESOLUTION)
```

The function `select\_start()` and `get\_goal\_pixels()` are not modified, and this is the end of `map\_processor.py`.

After this part, in `main.py`'s `main()`, I used the map_img_for_selection to display in selection.

```
print("=== Step 2: Selecting Agent Start and Goal Positions ===")
start = select_start(map_img_for_display)
goal_prompt, goal = pick_goal(map_img)
print(f"Goal pixel selected at coordinates: {goal}")
```

Part 2. RRT algorithm implementation

In this part, I wrote the main content in `plan\_path()` as the instruction. Here the returning values are:

- tree, parents: a list of nodes and a dictionary of child-parent pairs. This is from the basic RRT.
- leave_obstacle_path: a list of nodes in the RRT path, but adjusted to leave obstacles.
- simple_path: a list of nodes, which is last path's simplification.
- path: the final path, which is the last path adjusted to leave obstacles.

```
print("=== Step 3: Executing Path Planning (RRT) ===")
# ===== TODO 2 =====
# implement RRT path planner in plan_path()

path, simple_path, leave_obstacle_path, tree, parents = plan_path(
    start, goal_prompt, goal, occupancy_map, map_img)
if not path:
    print("Planner could not find a path.")
    sys.exit(1)
```

This function, `plan\_path()`, is written in `main.py`. Here is the preparation for RRT, where I use 0.1m steps and 0.4m radius tolerance. The code executes at most 50000 times before giving up.

```
# Note: Goal is a directional hint.
# We should stop once the surrounding meets goal_prompt
def plan_path(start, goal_prompt, goal, occupancy_map, map_img):
    MAX_ITER = 50000
    STEP_SIZE = 0.10 * MAP_RESOLUTION # 10cm per step
    GOAL_BIAS = 0.50 # 50% chance to directly explore
    towards the goal
    GOAL_DIST = 0.40 * MAP_RESOLUTION # 40cm radius to accept the goal

    height, width = occupancy_map.shape
    goal_color = np.array(SEMANTIC_DICTS["colors"][goal_prompt][0]) /
    255.0
    # the semantic color of the goal

    # preallocate np array as the tree
    tree_arr = np.zeros((MAX_ITER + 2, 2), dtype=float)
    tree_arr[0] = start
    num_nodes = 1

    # dict for parents (node index : parent node index)
    parents = {0: None}
```

Then, the beginning part of my `plan\_path()` follows the procedure of a basic RRT. First it randomly samples some points, find the nearest point, and try to step from that point.

```
print("Running RRT...")
for i in range(MAX_ITER):
    # 1. random sampling
    if random.random() < GOAL_BIAS:
        x_rand = goal
    else:
        x_rand = (random.uniform(0, width - 1), random.uniform(0, height - 1))

    # 2. find nearest node
    diffs = tree_arr[:num_nodes] - x_rand
    sq_dists = diffs[:, 0]**2 + diffs[:, 1]**2 # no sqrt makes it faster
    nearest_idx = int(np.argmin(sq_dists))
    x_nearest = tree_arr[nearest_idx]

    # 3. step from the nearest node
    d = dist(x_nearest, x_rand)
    if d <= STEP_SIZE:
        x_new = x_rand
    else:
        theta = np.arctan2(x_rand[1] - x_nearest[1], x_rand[0] - x_nearest[0])
        x_new = (
            float(x_nearest[0] + STEP_SIZE * np.cos(theta)),
            float(x_nearest[1] + STEP_SIZE * np.sin(theta))
        )
```

If there is a collision, give up the stepping and enter next round. Otherwise, the successful step is added to the tree.

```
# 4. collision check
if not is_collision_free(x_nearest, x_new, occupancy_map):
    continue

# 5. add node to tree array
new_idx = num_nodes
tree_arr[new_idx] = x_new
parents[new_idx] = nearest_idx
num_nodes += 1
```

And then, it checks if the goal is reached. I loosen the checking because the chosen goal point is not always "reachable" under the tolerance, but the same semantic-coloured points are often reachable.

```
# 6. check if goal is reached
# - the goal color should be in a rectangle centered at x_new

# 6.1. local map (a rectangle)
u_c, v_c = int(round(x_new[0])), int(round(x_new[1]))
r = int(GOAL_DIST)
u_min, u_max = max(0, u_c - r), min(width, u_c + r + 1)
v_min, v_max = max(0, v_c - r), min(height, v_c + r + 1)
local_map = map_img[v_min:v_max, u_min:u_max]

# 6.2. check if the goal color is in the local map
mask_goal = np.all(np.isclose(local_map, goal_color, atol=10/255.0), axis=-1)
if not np.any(mask_goal):
    continue
```

However, there is also a pitfall that some cushion points are on the desk in the study room. My way to mitigate this is to use the euclidean distance instead. In the end, the RRT path is generated. No matter what, I updates the goal point as new_goal.

```
# 6.3. check if the goal color is in euclidean distance
#       and select the closest goal color in local map as the new goal

goal_pixels_v, goal_pixels_u = np.where(mask_goal)

# Calculate euclidean distance from x_new to all found goal pixels
du = goal_pixels_u + u_min - x_new[0]
dv = goal_pixels_v + v_min - x_new[1]
dist_sq = du**2 + dv**2

min_idx = np.argmin(dist_sq)

# Square corners are > GOAL_DIST. Check if the closest point is truly within
the Euclidean circle
if dist_sq[min_idx] > GOAL_DIST**2:
    continue

# The exact closest point is the final landing goal
new_goal = (goal_pixels_u[min_idx] + u_min, goal_pixels_v[min_idx] + v_min)

# 7. once found, stop and form the reverted path
path = []
curr = new_idx
while curr is not None:
    path.append(tuple(tree_arr[curr]))
    curr = parents[curr]
path.reverse()
```

(Bonus) Extra Modification to Enhance RRT

(Continue from last section.) Then, I append the new goal point to the path because in the simulated world, this ensures the agent can be nearer and turn to the object. If it's in the physical world, a modification will be required.

```
# 7.1. append the new_goal in the end of the path
#       (the simulator will try to get the closest no matter reachable or not)
path.append(new_goal)
print(f"RRT Success! Found a path with {len(path)} steps.")

# 7.2 leave obstacles
leave_obstacle_path = path_leave_obstacles(path, occupancy_map,
mth_tengen_norm=False)
print(f"Path left obstacles.")

# 7.3. simplify the path
simple_path = simplify_path(path, occupancy_map)
print(f"Path simplified to {len(simple_path)} steps.")

# 7.4. leave obstacles
leave_obstacle_simple_path = path_leave_obstacles(simple_path, occupancy_map)
print(f"Path left obstacles.")

return leave_obstacle_simple_path, simple_path, leave_obstacle_path, tree_arr
[:num_nodes], parents
```

From the code above, I process the RRT path by moving away from obstacles → simplifying → moving away from obstacles again. Here are the functions:

(Bonus 1) Reduce the chance of collision

`path\_leave\_obstacles()` can move the nodes P_i in the direction of "norm", a normalized vector perpendicular to the "tangent line" (i.e., a vector from the previous node P_{i-1} to the next node P_{i+1}), and/or in a direction determined by the obstacle O_1 , a normalized vector that's $P_i - O_1$. It aims to reduce the chance that agents are blocked by the obstacles.

```
def path_leave_obstacles(path, occupancy_map, mth_tengen_norm=True,
mth_near_obs=True, repeat=5):
    if len(path) <= 3:
        return path

    height, width = occupancy_map.shape
    new_path = list(path)

    # helper: quick search for nearest obstacle
    def find_nearest_obstacle(p, max_r=30): ...

    # check if offset P_curr by t * N, is it collision free to P_prev and P_next
    def check_free(t, P_curr, N, P_prev, P_next): ...

    # find a segment containing the original point, and
    # all points except for the boundaries are collision free
    def boundary_search(P_curr, N, P_prev, P_next, step_dir): ...

    # do the whole path for five times
    for pass_idx in range(repeat):
        # don't move start point and the last two points
        for i in range(1, len(new_path) - 2):
            P_prev = new_path[i-1]
            P_curr = new_path[i]
            P_next = new_path[i+1]

            ## 1. move according to norm
            if mth_tengen_norm: ...

            ## 2. move leaving nearest obstacle
            if mth_near_obs: ...

    return new_path
```

For either direction, it finds a segment of movable range (i.e., in this range, the path to the last and next points are collision free), where the original unmoved situation is contained, and the boundaries are binary-searched to touch the "collision" situation.

```

# find a segment containing the original point, and
# all points except for the boundaries are collision free
def boundary_search(P_curr, N, P_prev, P_next, step_dir):
    t = 0.0
    step = 2.0 * step_dir

    # 1. exponential expansion: find an upper bound
    while check_free(t + step, P_curr, N, P_prev, P_next):
        t += step
        step *= 2.0
        if abs(t) > 500: # 防呆，不會真的跑無限遠
            break

    # 2. binary search: find the exact boundary
    low = t
    high = t + step
    while not abs(high - low) < 1: # ensure px level precision
        mid = (low + high) / 2.0
        if check_free(mid, P_curr, N, P_prev, P_next):
            low = mid # this mid is collision free
        else:
            high = mid # this mid is not collision free

    return low

```

```

## 1. move according to norm
if mth_tengen_norm:
    # tangent vector: (prev - next) point
    dx = P_next[0] - P_prev[0]
    dy = P_next[1] - P_prev[1]
    length = np.hypot(dx, dy)
    if length < 1e-3:
        continue

    # find the normal vector perpendicular to the tangent vector
    N = (-dy / length, dx / length)

    # find the boundaries (happen to be not collision free)
    t_pos = boundary_search(P_curr, N, P_prev, P_next, 1)
    t_neg = boundary_search(P_curr, N, P_prev, P_next, -1)

    # take the middle of the two boundaries as the new path point
    opt_t = (t_pos + t_neg) / 2.0
    new_path[i] = (P_curr[0] + opt_t * N[0], P_curr[1] + opt_t * N[1])
    P_curr = new_path[i] # Update P_curr from Step 1

```

This way, we can use the middle point in this segment as the new position for that point to reduce the chance of collision.

(Bonus 2) Simplify the Path to Speed Up Navigation

`simplify\_path()` is another function that enhance the result. An observation is that, shorter step-size in RRT makes it easier to go through complicated and tortuous pathways. However, in the simulation, the path formed by multiple steps with different directions causes a long time to stop and turning left and right.

To fix that, this function tries to reduce the extra nodes in broad areas. For each point P_i , it aggressively finds another node $P_j: j > i$ from the path end to the current node, to see if the connection from P_i to P_j is collision free. If it is, the nodes in between are skipped.

```
def simplify_path(path, occupancy_map):
    if len(path) <= 3:
        return path

    core_path = path[:-1] # keep the goal point
    simple_path = [core_path[0]]
    curr_idx = 0

    while curr_idx < len(core_path) - 1: # keep the last point to see the object
        # find the furthest node that can be connected to the current node
        for next_idx in range(len(core_path) - 1, curr_idx, -1):
            if is_collision_free(core_path[curr_idx], core_path[next_idx],
                                occupancy_map):
                simple_path.append(core_path[next_idx])
                curr_idx = next_idx
                break

    simple_path.append(path[-1])
    return simple_path
```

(Bonus 3) Rewritten of RRT with Numpy

This is not algorithmic improvements, but a reconsidered implementation. I was using Python's native list for the `tree` array, and use a for loop to calculate the nearest point on the tree. However, it significantly slows down (taking 3~5 seconds) for `MAX\_ITER = 10000`, and becomes unbearable when `MAX\_ITER = 50000`. The overhead should stem from the interpreter of code and the incontiguous memory allocation due to frequent `.append()`. Using a preallocated numpy array can ensure the contiguity of memory and thus utilize the spatial locality of cache, while the vectorized methods from numpy move the calculation to the native zone, preventing the overhead of interpreters.

Part 3. Robot navigation

Here is the code to visualize the planned navigation path. I used cv2 to draw the lines and nodes. The basic RRT's tree, the adjusted RRT path to move away from obstacles, the simplified of the previous path, and the adjusted one of the previous path to leave the obstacles are all drawn. Also, the destination is drawn as a black big circle.

```
print("=== Step 4: Visualizing the Planned Path ===")
# ===== TODO 3 =====
# Visualize the planned path over the map

vis_map = (np.copy(map_img_for_display) * 255).astype(np.uint8)
vis_map = np.ascontiguousarray(vis_map) # prevent cv2 complaining

# draw the RRT tree (light blue) and nodes (cyan)
for idx in range(1, len(tree)):
    parent_idx = parents[idx]
    if parent_idx is not None:
        pt_end = (int(tree[idx][0]), int(tree[idx][1]))
        pt_start = (int(tree[parent_idx][0]), int(tree[parent_idx][1]))
        cv2.line(vis_map, pt_start, pt_end, (255, 200, 150), 1)
        cv2.circle(vis_map, pt_end, 1, (255, 255, 0), -1)

# draw the leave obstacle path (blue line) and nodes (light green)
for i in range(len(leave_obstacle_path) - 1):
    pt1 = (int(leave_obstacle_path[i][0]), int(leave_obstacle_path[i][1]))
    pt2 = (int(leave_obstacle_path[i+1][0]), int(leave_obstacle_path[i+1][1]))
    cv2.line(vis_map, pt1, pt2, (255, 0, 0), 1)
    cv2.circle(vis_map, pt1, 1, (175, 255, 175), -1)

# draw the simple path (yellow line) and nodes (light green)
for i in range(len(simple_path) - 1):...

# draw the final path (red line) and nodes (green)
for i in range(len(path) - 1):...

# draw the goal point (black big circle)
new_goal = path[-1]
cv2.circle(vis_map, (int(new_goal[0]), int(new_goal[1])), 2, (0, 0, 0), -1)

cv2.imshow("RRT Found Path", vis_map)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Then it's about the navigation in simulation. I didn't heavily modify navigator.py, only reduced the MOVE_AMOUNT to 0.05 (m). Here is the `main()`'s TODO 4 in `main.py`.

```
print("=== Step 5: Translating Path to Habitat Simulator ===")
# ===== TODO 4 =====
# Convert pixel path to world coordinates
# world_path is a list of tuples(float, float) representing waypoints in world coordinates

min_u, min_v, map_resolution = trans_info
world_path = []

for (u, v) in path:
    x_world = (u / map_resolution) + min_u
    z_world = (v / map_resolution) + min_v
    world_path.append((x_world, z_world))

run_in_sim(world_path[0], world_path, goal_prompt)
```

This is my semantic dictionary and `run\_in\_sim()` function. I tried to solve the multiple cushion problem - they are listed in the mentioned JSON file, but I couldn't understand the coordinate system in that file. In the end, I decided to hard-code the most often goal point as the index (id) for cushion. The abandoned code is collapsed in the function.

```
# Sample semantic color and index dictionaries for a few object categories.
# Check hw0/replica_v1/apartment_0/habitat/info_semantic.json and
# hw3/color_coding_semantic_segmentation_classes.xlsx for the full list of
# categories and their corresponding colors and indices.
SEMANTIC_DICTS = {
    "colors": {
        "rack": [[0, 255, 133]],
        "cooktop": [[7, 255, 224]],
        "sofa": [[10, 0, 255]],
        "cushion": [[255, 9, 92]],
        "stair": [[173, 255, 0]]
    },
    "indices": {
        "rack": 8,
        "cooktop": 280,
        "sofa": 196,
        "cushion": 430,
        "stair": 192
    },
}
```

```
def run_in_sim(start_world: Tuple[float, float], world_path: List[Tuple[float, float]], goal_prompt: str):
    start_x, start_z = start_world
    print(f"Spawning Agent at world position: ({start_x:.3f}, {start_z:.3f})")

    sim, agent, _ = init_sim(start_x=start_x, start_z=start_z)

    id = SEMANTIC_DICTS["indices"][goal_prompt]
    # abandoned since the coordination mapping is weird

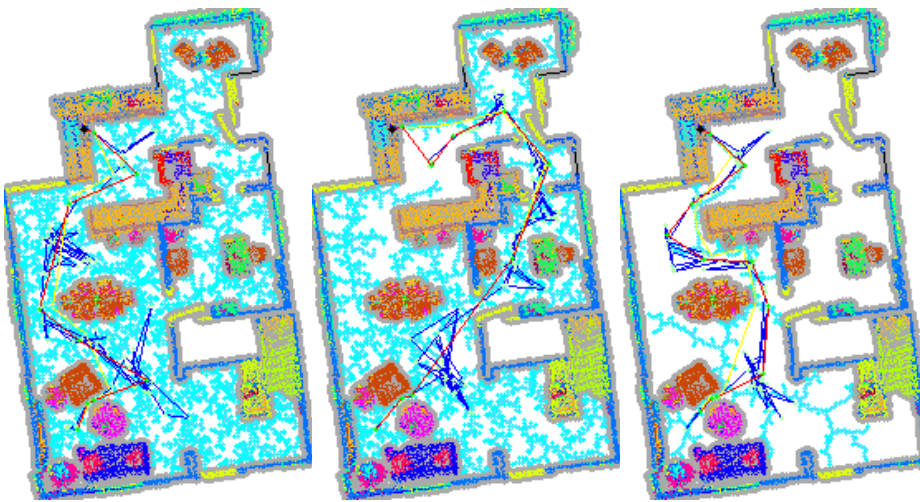
    { ...

    execute_waypoint_path(world_path, sim, agent, id)
```

Result and Discussion

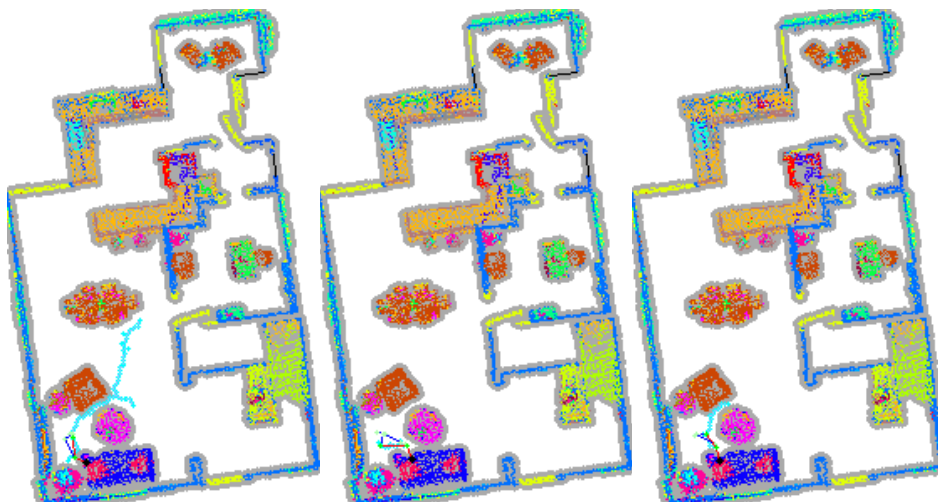
Here are the ``path_plan()`` function's outputs.

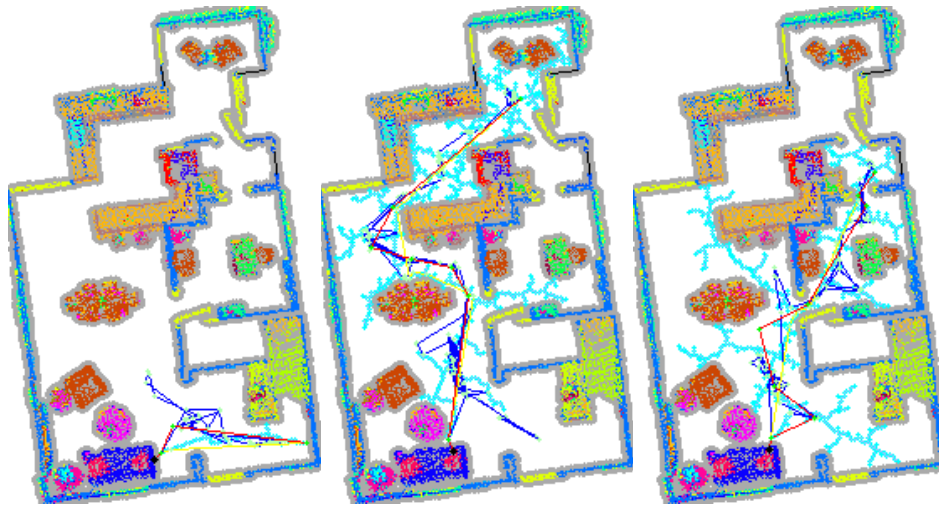
- Light Blue lines and Cyan points are the RRT tree.
- Blue lines and Light Green points are the RRT path adjusted to leave obstacles.
- Yellow lines and Light Green points are the path simplified from last path.
- Red lines and Green points are the final path, which is the last path adjusted again to leave obstacles.
- The Black big dot are the goal points.
- Grey is the inflation of obstacles.



The left three starts from the narrow area to the cooktop. Thanks to the numpy optimization (in Bonus 3), they all plan the path in seconds. The path can go through the tortuous pathway due to small step size (= 10cm).

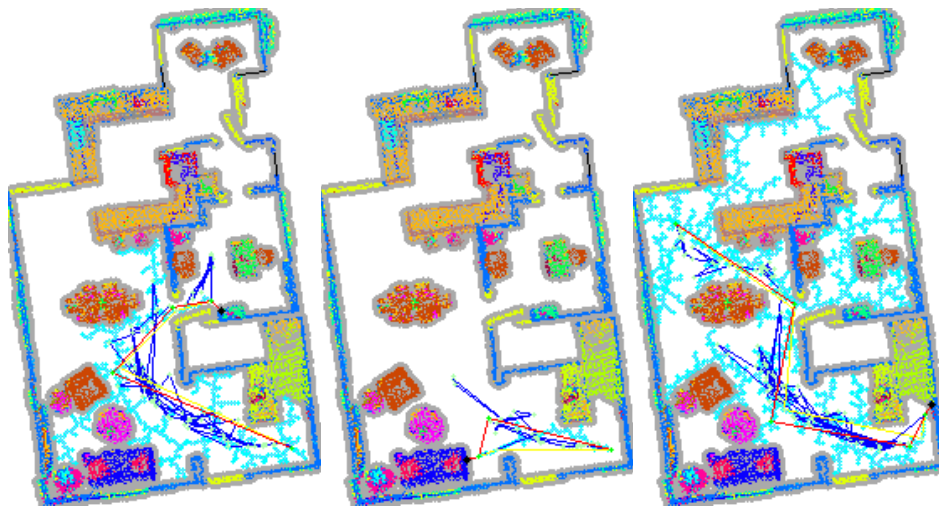
In my ``path_plan()``, the destination it finds is often the nearest instance of the class. For example, the below executions set "cushion" as the destinations.





Also, we can observe from the above. With the path simplification feature (Bonus 2), 100~200 steps can often be reduced to < 15 steps, which significantly boosts the simulation. For each execution above, path planning takes no more than 5 seconds.

To prove rack, sofa and stair can also be reached, here are the screenshots:



My code used to face the problem that, although a path is generated and looks good on the map, but due to the gap between our map and the simulation, it still fails. However, with my ``path_leave_obstacles()`` function (Bonus 1), such a situation is rare to meet again. It can be observed that blue and red lines often have more spaces than the light blue and yellow lines.

Reference

The main reference is "Lecture 7 - path planning.pdf" from our course. Also, I talked to Gemini (Antigravity) for some implementation details. There are no other references.

2. Questions

Q1. In the RRT algorithm, you can adjust the step size and bias (the number balance between exploration and exploitation). Please explain how the two numbers affect the RRT sampling result.

A higher step size can significantly increase the speed of RRT, but it is not good for going through complicated and narrow pathways. A smaller step size can fix this by generating multiple short steps to go through that pathway, but it also takes much longer time when the starting point is far away.

A higher bias creates a more straight path when there is no obstacles from the current point to the goal point. However, when RRT needs to bypass obstacles, it has a hard time struggling, and this also causes the RRT prone to generate a path along the edge of obstacles. Besides, although a higher bias boost the exploration when it's near to the goal point with less obstacles around, it makes the exploration really slow and often give up due to max iteration reach when the starting point is far away with many obstacles in between.

Q2. If you want to apply the indoor navigation pipeline in the real world, what problems may you encounter?

First, how to efficiently do SLAM and build a map? HW2 provides an example for 3D space (point cloud) reconstruction from photos and, but what I wrote is not good enough to put into the real world, because the point clouds often fail to align, especially when entering narrow spaces like the bathrooms, or looking at featureless objects like the walls. SLAM algorithm may also need to know what are the moving elements (like human objects) that may change in the map.

Second, how to do semantic/instance/panoptic segmentation, and label them with some identifiers? A good model and some techniques may be required, such that the agent knows which pillow to get, instead of randomly choosing once of them.

Third, what if the robot falls down? As discussed above, the inflation of obstacles (configuration space) may not always be accurate due to the low quality construction of obstacle map. Then, the robot may bump into the wall, accidentally make a cup fall, or even stumble due to the floor removal in my implementation (which uses the height to cut).

In the end, how to compensate for errors? The pipeline cannot be directly used in the physical world where offsets are generated everywhere. The robot can hardly follow our instruction to turn to a direction and walk for a distance

precisely. Thus, there must be something like SLAM (or at least, registration from HW2) or other indoor localization infrastructures like UWB.